

Министерство науки и высшего образования РФ
Федеральное государственное автономное образовательное учреждение
высшего профессионального образования
Национальный исследовательский технологический университет "МИСИС"

Институт компьютерных наук (ИKN)

Кафедра инфокоммуникационных технологий (ИКТ)

Отчет по лабораторной работе № 9
по дисциплине «Базы данных»
на тему «Проектирование баз данных»

Выполнил:
студент группы БПИ-24-3

Паненкова Е.Г.

Проверил:
доцент каф. ИКТ
Стучилин В.В.

Москва, 2025

Цель: научиться создавать базу данных на инфологическом, даталогическом и физическом этапах проектирования, разрабатывать триггеры и хранимые процедуры/функции, обеспечивать разграничения прав пользователей.

Задание:

Спроектировать БД по следующим требованиям:

1. БД должна содержать не менее 7-ми таблиц. Из них не менее 6-ти должны быть связаны между собой.
2. Все таблицы необходимо заполнить (не менее трех записей).
3. БД должна содержать не менее 3-х триггеров.
4. БД должна содержать не менее 3-х хранимых процедур/функций.
5. Доступ к БД должны иметь не менее 3-х пользователей с разными правами (кроме root).

База данных:

Криминальная картотека: серийные преступники

Теоретическая часть.

Проектирование баз данных - это процесс создания модели базы данных, который включает три основных этапа:

1. Инфологический этап - создание концептуальной модели, независимой от СУБД
2. Даталогический этап - преобразование в логическую модель с учетом выбранной СУБД
3. Физический этап - реализация модели в конкретной СУБД

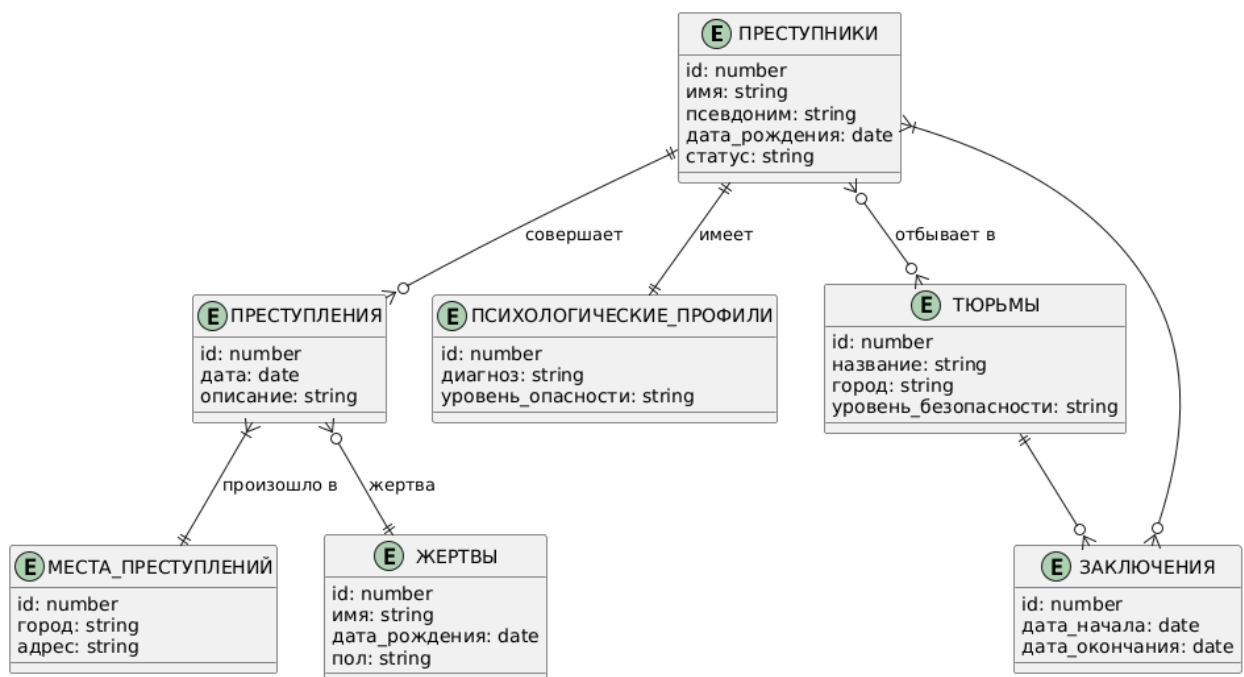
Нормализация - процесс устранения избыточности данных и аномалий обновления:

- 1НФ: Все атрибуты атомарны, нет повторяющихся групп
- 2НФ: Выполнена 1НФ + каждый неключевой атрибут полно зависит от первичного ключа
- 3НФ: Выполнена 2НФ + отсутствуют транзитивные зависимости

Триггеры - хранимые программы, автоматически выполняемые при наступлении определенных событий (INSERT, UPDATE, DELETE).

Хранимые процедуры/функции - именованные наборы SQL-инструкций, хранящиеся на сервере.

Инфологическая модель.



Типы связей:

- Преступник → Преступления (1:M)
- Преступление → Место преступления (1:1)
- Преступление → Жертва (M:1)
- Преступник → Психологический профиль (1:1)
- Преступник → Тюрьма (M:M через Заключения)

Даталогическая модель (1НФ, 2НФ)

Приведение к 1НФ:

Все таблицы приведены к первой нормальной форме:

- Отсутствуют повторяющиеся группы данных
- Все атрибуты атомарны (неделимы)
- В каждом столбце хранятся данные одного типа

- Отсутствуют массивы и списки

Ненормализованная запись:

Преступник(имя, [жертва1, жертва2, жертва3])

1НФ:

Преступник(имя)

Преступления(преступник_id, жертва_id)

Жертвы(имя)

Приведение к 2НФ:

Все таблицы приведены ко второй нормальной форме:

- Выполнена 1НФ
- Созданы правильные первичные ключи
- Каждый неключевой атрибут функционально полно зависит от первичного ключа

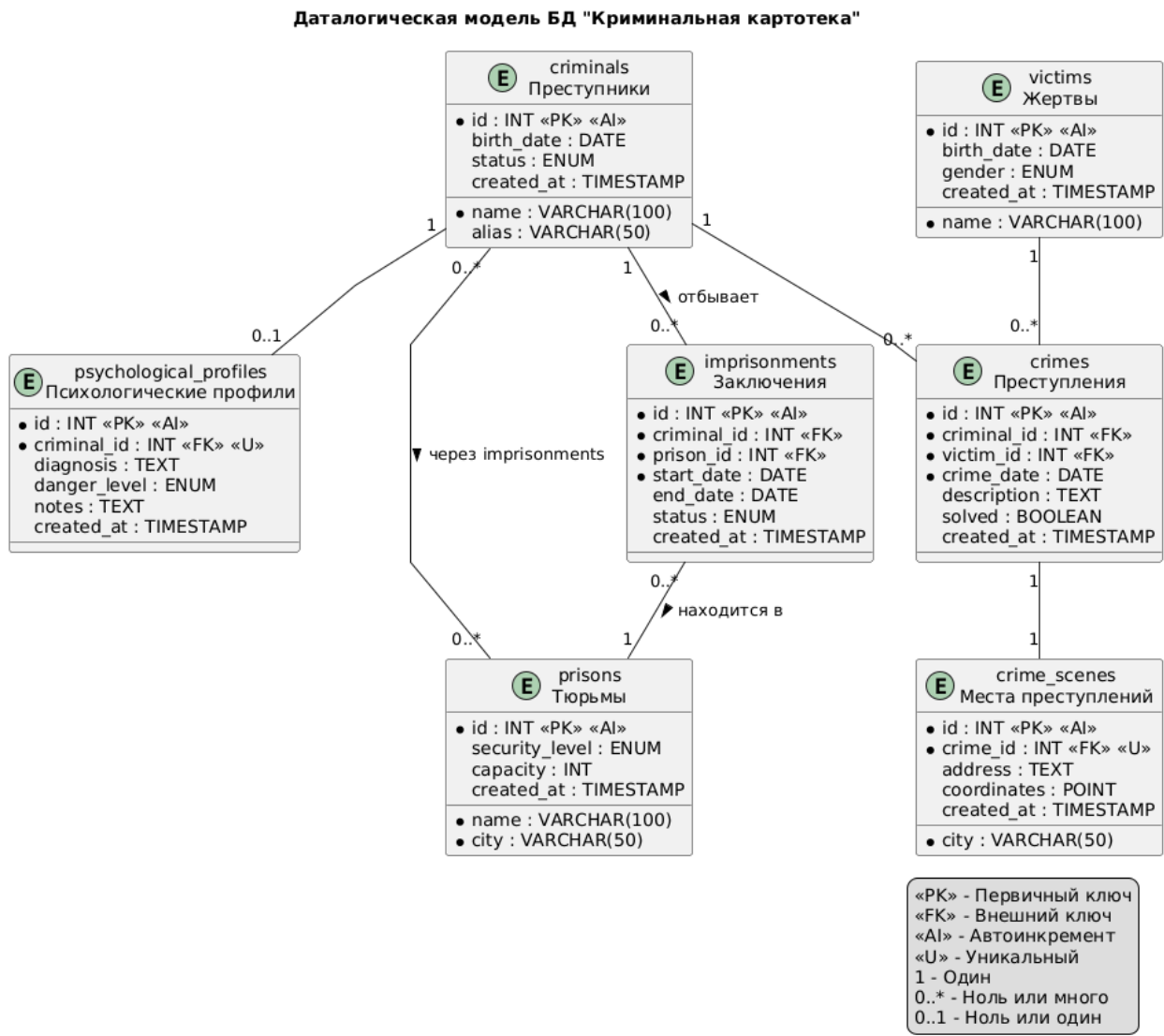
Проверка 2НФ для таблицы crimes:

- Первичный ключ: id
- Атрибуты: criminal_id, victim_id, crime_date, description
- Все неключевые атрибуты зависят от полного ключа id

Проверка 2НФ для таблицы imprisonments:

- Первичный ключ: id
- Атрибуты: criminal_id, prison_id, start_date, end_date
- Все атрибуты зависят от полного ключа id

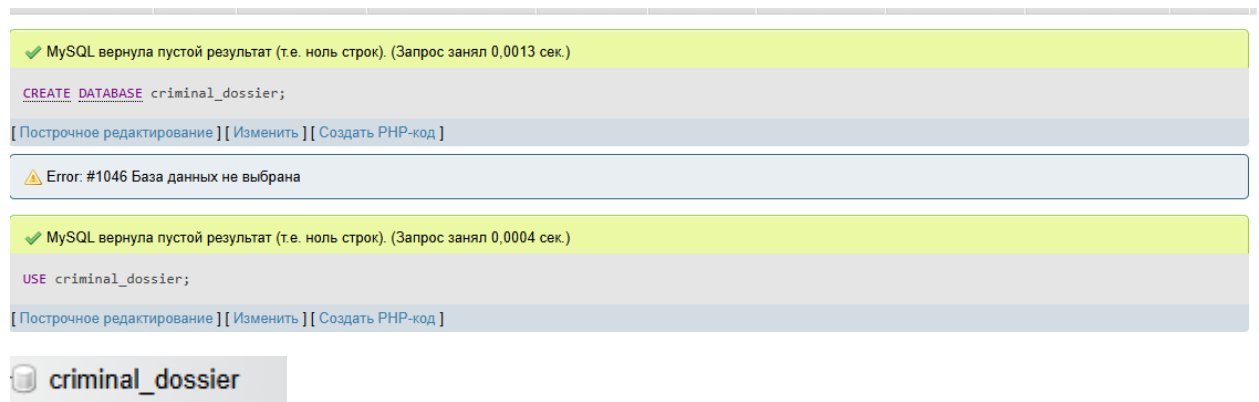
Физическая модель (структура БД и всех таблиц)



Создание базы данных:

```
CREATE DATABASE criminal_dossier;
```

```
USE criminal_dossier;
```



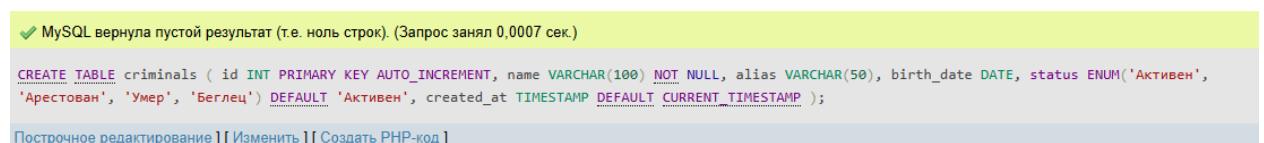
The screenshot shows a MySQL command-line interface with a light blue background. It displays two successful SQL queries and their results. The first query is `CREATE DATABASE criminal_dossier;` and the second is `USE criminal_dossier;`. Both queries returned an empty result set. Below the queries, there is a section titled `criminal_dossier` with a database icon.

```
✓ MySQL вернула пустой результат (т.е. ноль строк). (Запрос занял 0,0013 сек.)  
CREATE DATABASE criminal_dossier;  
[ Построчное редактирование ] [ Изменить ] [ Создать PHP-код ]  
Error: #1046 База данных не выбрана  
✓ MySQL вернула пустой результат (т.е. ноль строк). (Запрос занял 0,0004 сек.)  
USE criminal_dossier;  
[ Построчное редактирование ] [ Изменить ] [ Создать PHP-код ]  
criminal_dossier
```

Структура таблиц:

1. Таблица criminals (Преступники):

```
CREATE TABLE criminals (  
    id INT PRIMARY KEY AUTO_INCREMENT,  
    name VARCHAR(100) NOT NULL,  
    alias VARCHAR(50),  
    birth_date DATE,  
    status ENUM('Активен', 'Арестован', 'Умер', 'Беглец') DEFAULT 'Активен',  
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP  
);
```

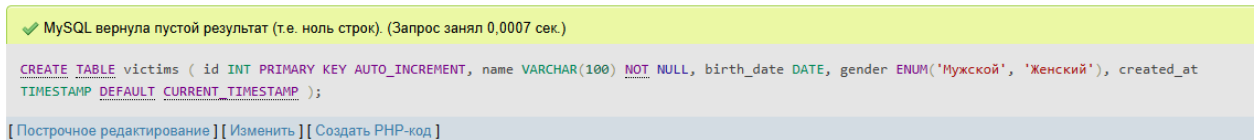


The screenshot shows a MySQL command-line interface with a light blue background. It displays a successful SQL query to create a table named 'criminals'. The query is `CREATE TABLE criminals ( id INT PRIMARY KEY AUTO_INCREMENT, name VARCHAR(100) NOT NULL, alias VARCHAR(50), birth_date DATE, status ENUM('Активен', 'Арестован', 'Умер', 'Беглец') DEFAULT 'Активен', created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP );`. The query returned an empty result set. Below the query, there is a section titled `criminals` with a table icon.

```
✓ MySQL вернула пустой результат (т.е. ноль строк). (Запрос занял 0,0007 сек.)  
CREATE TABLE criminals ( id INT PRIMARY KEY AUTO_INCREMENT, name VARCHAR(100) NOT NULL, alias VARCHAR(50), birth_date DATE, status ENUM('Активен', 'Арестован', 'Умер', 'Беглец') DEFAULT 'Активен', created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP );  
[ Построчное редактирование ] [ Изменить ] [ Создать PHP-код ]
```

2. Таблица victims (Жертвы):

```
CREATE TABLE victims (  
    id INT PRIMARY KEY AUTO_INCREMENT,  
    name VARCHAR(100) NOT NULL,  
    birth_date DATE,  
    gender ENUM('Мужской', 'Женский'),  
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP  
);
```



3. Таблица crimes (Преступления):

```
CREATE TABLE crimes (  
    id INT PRIMARY KEY AUTO_INCREMENT,  
    criminal_id INT NOT NULL,  
    victim_id INT NOT NULL,  
    crime_date DATE NOT NULL,  
    description TEXT,  
    solved BOOLEAN DEFAULT FALSE,  
    FOREIGN KEY (criminal_id) REFERENCES criminals(id) ON DELETE  
    CASCADE,  
    FOREIGN KEY (victim_id) REFERENCES victims(id) ON DELETE  
    CASCADE,  
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP  
);
```

✔ MySQL вернула пустой результат (т.е. ноль строк). (Запрос занял 0,0005 сек.)

```
CREATE TABLE crimes ( id INT PRIMARY KEY AUTO_INCREMENT, criminal_id INT NOT NULL, victim_id INT NOT NULL, crime_date DATE NOT NULL, description TEXT, solved BOOLEAN DEFAULT FALSE, FOREIGN KEY (criminal_id) REFERENCES criminals(id) ON DELETE CASCADE, FOREIGN KEY (victim_id) REFERENCES victims(id) ON DELETE CASCADE, created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP );
```

[\[Построчное редактирование \]](#) [\[Изменить \]](#) [\[Создать PHP-код \]](#)

4. Таблица psychological_profiles (Психологические профили):

```
CREATE TABLE psychological_profiles (  
    id INT PRIMARY KEY AUTO_INCREMENT,  
    criminal_id INT UNIQUE NOT NULL,  
    diagnosis TEXT,  
    danger_level ENUM('Низкий', 'Средний', 'Высокий', 'Критический')  
    DEFAULT 'Средний',  
    notes TEXT,  
    FOREIGN KEY (criminal_id) REFERENCES criminals(id) ON DELETE  
    CASCADE,  
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP  
);
```

✔ MySQL вернула пустой результат (т.е. ноль строк). (Запрос занял 0,0004 сек.)

```
CREATE TABLE psychological_profiles ( id INT PRIMARY KEY AUTO_INCREMENT, criminal_id INT UNIQUE NOT NULL, diagnosis TEXT, danger_level ENUM('Низкий', 'Средний', 'Высокий', 'Критический') DEFAULT 'Средний', notes TEXT, FOREIGN KEY (criminal_id) REFERENCES criminals(id) ON DELETE CASCADE, created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP );
```

[\[Построчное редактирование \]](#) [\[Изменить \]](#) [\[Создать PHP-код \]](#)

5. Таблица crime_scenes (Места преступлений):

```
CREATE TABLE crime_scenes (  
    id INT PRIMARY KEY AUTO_INCREMENT,  
    crime_id INT UNIQUE NOT NULL,  
    city VARCHAR(50) NOT NULL,
```

```
address TEXT,  
coordinates POINT,  
FOREIGN KEY (crime_id) REFERENCES crimes(id) ON DELETE  
CASCADE,  
created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP  
);
```

✓ MySQL вернула пустой результат (т.е. ноль строк). (Запрос занял 0,0004 сек.)

```
CREATE TABLE crime_scenes ( id INT PRIMARY KEY AUTO_INCREMENT, crime_id INT UNIQUE NOT NULL, city VARCHAR(50) NOT NULL, address TEXT, coordinates POINT,  
FOREIGN KEY (crime_id) REFERENCES crimes(id) ON DELETE CASCADE, created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP );
```

[Построчное редактирование] [Изменить] [Создать PHP-код]

6. Таблица prisons (Тюрьмы):

```
CREATE TABLE prisons (  
id INT PRIMARY KEY AUTO_INCREMENT,  
name VARCHAR(100) NOT NULL,  
city VARCHAR(50) NOT NULL,  
security_level ENUM('Минимальный', 'Средний', 'Максимальный',  
'Сверхмаксимальный') DEFAULT 'Средний',  
capacity INT,  
created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP  
);
```

✓ MySQL вернула пустой результат (т.е. ноль строк). (Запрос занял 0,0006 сек.)

```
CREATE TABLE prisons ( id INT PRIMARY KEY AUTO_INCREMENT, name VARCHAR(100) NOT NULL, city VARCHAR(50) NOT NULL, security_level ENUM('Минимальный',  
'Средний', 'Максимальный', 'Сверхмаксимальный') DEFAULT 'Средний', capacity INT, created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP );
```

[Построчное редактирование] [Изменить] [Создать PHP-код]

7. Таблица imprisonments (Заключения):

```
CREATE TABLE imprisonments (  
id INT PRIMARY KEY AUTO_INCREMENT,
```

```

criminal_id INT NOT NULL,
prison_id INT NOT NULL,
start_date DATE NOT NULL,
end_date DATE,
status ENUM('Отбывает', 'Освобожден', 'Переведен') DEFAULT 'Отбывает',
FOREIGN KEY (criminal_id) REFERENCES criminals(id) ON DELETE
CASCADE,
FOREIGN KEY (prison_id) REFERENCES prisons(id) ON DELETE
CASCADE,
created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);

```

✓ MySQL вернула пустой результат (т.е. ноль строк). (Запрос занял 0,0005 сек.)

```

CREATE TABLE imprisonments ( id INT PRIMARY KEY AUTO_INCREMENT, criminal_id INT NOT NULL, prison_id INT NOT NULL, start_date DATE NOT NULL, end_date
DATE, status ENUM('Отбывает', 'Освобожден', 'Переведен') DEFAULT 'Отбывает', FOREIGN KEY (criminal_id) REFERENCES criminals(id) ON DELETE CASCADE,
FOREIGN KEY (prison_id) REFERENCES prisons(id) ON DELETE CASCADE, created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP );

```

[Построение редактирование] [Изменить] [Создать PHP-код]



Заполнение таблиц

-- Заполнение таблицы criminals (Преступники)

```

INSERT INTO criminals (name, alias, birth_date, status) VALUES
('Теодор Качинский', 'Унабомбер', '1942-05-22', 'Арестован'),

```

('Джеффри Дамер', 'Милуокский каннибал', '1960-05-21', 'Умер'),
('Андрей Чикатило', 'Ростовский потрошитель', '1936-10-16', 'Умер'),
('Чарльз Мэнсон', NULL, '1934-11-12', 'Умер'),
('Джон Уэйн Гейси', 'Клоун-убийца', '1942-03-17', 'Умер'),
('Ричард Рамирес', 'Ночной Сталкер', '1960-02-29', 'Умер');

-- Заполнение таблицы victims (Жертвы)

```
INSERT INTO victims (name, birth_date, gender) VALUES
```

('Хью Скроггинс', '1955-08-12', 'Мужской'),
('Стивен Эйвери', '1962-07-09', 'Мужской'),
('Лена Загортова', '1971-03-15', 'Женский'),
('Шэрон Тейт', '1943-01-24', 'Женский'),
('Роберт Пирст', '1955-11-03', 'Мужской'),
('Мария Эрнандес', '1965-08-22', 'Женский'),
('Джеймс Хьюстон', '1948-12-30', 'Мужской'),
('Лариса Кузнецова', '1968-07-18', 'Женский'),
('Эйб Вайнштейн', '1958-04-11', 'Мужской');

-- Заполнение таблицы crimes (Преступления)

```
INSERT INTO crimes (criminal_id, victim_id, crime_date, description, solved)
VALUES
```

(1, 1, '1978-05-25', 'Взрыв посылки в Северо-Западном университете', TRUE),
(1, 2, '1985-12-11', 'Взрыв в компьютерном магазине Сакраменто', TRUE),
(2, 3, '1978-06-18', 'Первое убийство - автостопщик Стивен Хикс', TRUE),
(2, 4, '1991-03-25', 'Убийство и расчленение в квартире', TRUE),
(3, 5, '1978-12-22', 'Первая доказанная жертва - Лариса Савенко', TRUE),
(3, 6, '1982-09-15', 'Убийство в лесополосе', TRUE),
(4, 7, '1969-08-09', 'Убийство Шэрон Тейт и гостей', TRUE),
(5, 8, '1972-01-02', 'Первое убийство - Тимоти Маккой', TRUE),

(6, 9, '1984-06-28', 'Первое убийство - Дженни Винсов', TRUE);

-- Заполнение таблицы psychological_profiles (Психологические профили)

INSERT INTO psychological_profiles (criminal_id, diagnosis, danger_level, notes) VALUES

(1, 'Параноидная шизофрения, социопатия', 'Высокий', 'Гениальный математик, ненавидит технологии'),

(2, 'Некрофилия, каннибализм, пограничное расстройство личности', 'Критический', 'Собирал части тел жертв'),

(3, 'Сексуальный садизм, импотенция, психопатия', 'Критический', 'Испытывал оргазм только при убийствах'),

(4, 'Антисоциальное расстройство личности, мессианский бред', 'Высокий', 'Создал секту "Семья"'),

(5, 'Антисоциальное расстройство личности, бисексуальность', 'Высокий', 'Работал клоуном на детских праздниках'),

(6, 'Сатанизм, психопатия, наркомания', 'Критический', 'Имел татуировки в виде pentagram');

-- Заполнение таблицы prisons (Тюрьмы)

INSERT INTO prisons (name, city, security_level, capacity) VALUES

('Тюрьма ADX Флоренс', 'Флоренс, Колорадо', 'Сверхмаксимальный', 500),

('Тюрьма Колумбайн', 'Кэнон-Сити, Колорадо', 'Максимальный', 800),

('Кресты', 'Санкт-Петербург', 'Максимальный', 3000),

('Тюрьма Сан-Квентин', 'Сан-Квентин, Калифорния', 'Максимальный', 4000),

('Тюрьма Стейтвилл', 'Крест-Хилл, Иллинойс', 'Максимальный', 2000),

('Тюрьма Синг-Синг', 'Оссининг, Нью-Йорк', 'Максимальный', 1700);

-- Заполнение таблицы crime_scenes (Места преступлений)

INSERT INTO crime_scenes (crime_id, city, address, coordinates) VALUES

(1, 'Чикаго', 'Северо-Западный университет', POINT(41.8819, -87.6278)),

```
(2, 'Сакраменто', 'Компьютерный магазин на Ранчо-Кордова', POINT(38.5816, -121.4944)),
(3, 'Милуоки', 'Шоссе I-94', POINT(43.0389, -87.9065)),
(4, 'Милуоки', 'Апартаменты на Оксфорд-авеню', POINT(43.0746, -87.8843)),
(5, 'Ростов-на-Дону', 'Лесополоса возле аэропорта', POINT(47.2582, 39.7902)),
(6, 'Шахты', 'Окраина города', POINT(47.7085, 40.2140)),
(7, 'Лос-Анджелес', 'Вилла на Съело-драйв 10050', POINT(34.0928, -118.4177)),
(8, 'Чикаго', 'Норвуд-Парк', POINT(41.9631, -87.8152)),
(9, 'Лос-Анджелес', 'Глендейл', POINT(34.1425, -118.2551));
```

-- Заполнение таблицы imprisonments (Заключения)

```
INSERT INTO imprisonments (criminal_id, prison_id, start_date, end_date, status)
VALUES
```

```
(1, 1, '1998-05-04', NULL, 'Отбывает'),
(2, 2, '1992-07-23', '1994-11-28', 'Умер в тюрьме'),
(3, 3, '1994-02-15', '1994-02-15', 'Казнен'),
(4, 4, '1971-01-25', '2017-11-19', 'Умер в тюрьме'),
(5, 5, '1980-03-13', '1994-05-10', 'Казнен'),
(6, 6, '1989-11-07', '2013-06-07', 'Умер в тюрьме');
```

✓ Добавлено 6 строк.

Идентификатор вставленной строки: 6 (Запрос занял 0.0025 сек.)

```
-- Заполнение таблицы criminals (Преступники) INSERT INTO criminals (name, alias, birth_date, status) VALUES ('Теодор Качинский', 'Унабомбер', '1942-05-22', 'Арестован'), ('Джеффри Дамер', 'Милуокский каннибал', '1960-05-21', 'Умер'), ('Андрей Чикатило', 'Ростовский потрошитель', '1936-10-16', 'Умер'), ('Чарльз Мэнсон', NULL, '1934-11-12', 'Умер'), ('Джон Уэйн Гейси', 'Клоун-убийца', '1942-03-17', 'Умер'), ('Ричард Рамирес', 'Ночной Сталкер', '1960-02-29', 'Умер');
```

[\[Построчное редактирование\]](#) [\[Изменить\]](#) [\[Создать PHP-код\]](#)

✓ Добавлено 9 строк.

Идентификатор вставленной строки: 9 (Запрос занял 0.0005 сек.)

```
-- Заполнение таблицы victims (Жертвы) INSERT INTO victims (name, birth_date, gender) VALUES ('Хью Скромгинс', '1955-08-12', 'Мужской'), ('Стивен Эйвери', '1962-07-09', 'Мужской'), ('Лена Загирова', '1971-03-15', 'Женский'), ('Шэрон Тейт', '1943-01-24', 'Женский'), ('Роберт Лирст', '1955-11-03', 'Мужской'), ('Мария Эрнандес', '1965-08-22', 'Женский'), ('Джеймс Хьюстон', '1948-12-30', 'Мужской'), ('Лариса Кузнецова', '1968-07-18', 'Женский'), ('Эйб Вайнштейн', '1958-04-11', 'Мужской');
```

[\[Построчное редактирование\]](#) [\[Изменить\]](#) [\[Создать PHP-код\]](#)

✓ Добавлено 9 строк.

Идентификатор вставленной строки: 9 (Запрос занял 0,0005 сек.)

```
-- Заполнение таблицы crimes (Преступления) INSERT INTO crimes (criminal_id, victim_id, crime_date, description, solved) VALUES (1, 1, '1978-05-25', 'Взрыв посылки в Северо-Западном университете', TRUE), (1, 2, '1985-12-11', 'Взрыв в компьютерном магазине Сакраменто', TRUE), (2, 3, '1978-06-18', 'Первое убийство - автостопщик Стивен Хикс', TRUE), (2, 4, '1991-03-25', 'Убийство и расчленение в квартире', TRUE), (3, 5, '1978-12-22', 'Первая доказанная жертва - Лариса Савенко', TRUE), (3, 6, '1982-09-15', 'Убийство в лесополосе', TRUE), (4, 7, '1969-08-09', 'Убийство Шэрон Тейт и гостей', TRUE), (5, 8, '1972-01-02', 'Первое убийство - Тимоти Маккой', TRUE), (6, 9, '1984-06-28', 'Первое убийство - Джени Винсов', TRUE);
```

[Построчное редактирование] [Изменить] [Создать PHP-код]

✓ Добавлено 6 строк.

Идентификатор вставленной строки: 6 (Запрос занял 0,0005 сек.)

```
-- Заполнение таблицы psychological_profiles (Психологические профили) INSERT INTO psychological_profiles (criminal_id, diagnosis, danger_level, notes) VALUES (1, 'Параноидная шизофрения, социопатия', 'Высокий', 'Гениальный математик, ненавидит технологии'), (2, 'Некрофилия, канибализм, пограничное расстройство личности', 'Критический', 'Собирал части тел жертв'), (3, 'Сексуальный садизм, импотенция, психопатия', 'Критический', 'Испытывал оргазм только при убийствах'), (4, 'Антисоциальное расстройство личности, мессианский бред', 'Высокий', 'Создал секту "Семья"'), (5, 'Антисоциальное расстройство личности, бисексуальность', 'Высокий', 'Работал клоуном на детских праздниках'), (6, 'Сатанизм, психопатия, наркомания', 'Критический', 'Имел татуировки в виде pentagram');
```

[Построчное редактирование] [Изменить] [Создать PHP-код]

✓ Добавлено 6 строк.

Идентификатор вставленной строки: 6 (Запрос занял 0,0005 сек.)

```
-- Заполнение таблицы prisons (Тюрьмы) INSERT INTO prisons (name, city, security_level, capacity) VALUES ('Тюрьма ADX Флоренс', 'Флоренс, Колорадо', 'Сверхмаксимальный', 500), ('Тюрьма Колумбайн', 'Кэнон-Сити, Колорадо', 'Максимальный', 800), ('Кресты', 'Санкт-Петербург', 'Максимальный', 3000), ('Тюрьма Сан-Квентин', 'Сан-Квентин, Калифорния', 'Максимальный', 4000), ('Тюрьма Стейтвилл', 'Крест-Хилл, Иллинойс', 'Максимальный', 2000), ('Тюрьма Синг-Синг', 'Оссининг, Нью-Йорк', 'Максимальный', 1700);
```

[Построчное редактирование] [Изменить] [Создать PHP-код]

✓ Добавлено 9 строк.

Идентификатор вставленной строки: 9 (Запрос занял 0,0004 сек.)

```
-- Заполнение таблицы crime_scenes (Места преступлений) INSERT INTO crime_scenes (crime_id, city, address, coordinates) VALUES (1, 'Чикаго', 'Северо-Западный университет', POINT(41.8819, -87.6278)), (2, 'Сакраменто', 'Компьютерный магазин на Ранчо-Кордова', POINT(38.5816, -121.4944)), (3, 'Милуоки', 'Шоссе I-94', POINT(43.0389, -87.9065)), (4, 'Милуоки', 'Апартаменты на Оксфорд-авеню', POINT(43.0746, -87.8843)), (5, 'Ростов-на-Дону', 'Лесополоса возле аэропорта', POINT(47.2582, 39.7902)), (6, 'Шахты', 'Окраина города', POINT(47.7085, 40.2140)), (7, 'Лос-Анджелес', 'Вилла на Сьело-драйв 10050', POINT(34.0928, -118.4177)), (8, 'Чикаго', 'Норвуд-Парк', POINT(41.9631, -87.8152)), (9, 'Лос-Анджелес', 'Глендейл', POINT(34.1425, -118.2551));
```

[Построчное редактирование] [Изменить] [Создать PHP-код]

✓ Добавлено 6 строк.

Идентификатор вставленной строки: 6 (Запрос занял 0,0005 сек.)

```
-- Заполнение таблицы imprisonments (Заклечения) INSERT INTO imprisonments (criminal_id, prison_id, start_date, end_date, status) VALUES (1, 1, '1998-05-04', NULL, 'Отбывает'), (2, 2, '1992-07-23', '1994-11-28', 'Умер в тюрьме'), (3, 3, '1994-02-15', '1994-02-15', 'Казнен'), (4, 4, '1971-01-25', '2017-11-19', 'Умер в тюрьме'), (5, 5, '1980-03-13', '1994-05-10', 'Казнен'), (6, 6, '1989-11-07', '2013-06-07', 'Умер в тюрьме');
```

[Построчное редактирование] [Изменить] [Создать PHP-код]

Триггеры для базы данных:

1. Триггер для проверки даты преступления (не может быть в будущем)

```
DELIMITER //
```

```
CREATE TRIGGER before_crime_date_check
BEFORE INSERT ON crimes
FOR EACH ROW
BEGIN
    IF NEW.crime_date > CURDATE() THEN
        SIGNAL SQLSTATE '45000'
        SET MESSAGE_TEXT = 'Дата преступления не может быть в будущем';
    END IF;
END//
```

```
DELIMITER ;
```

✓ MySQL вернула пустой результат (т.е. ноль строк). (Запрос занял 0,0144 сек.)

```
CREATE TRIGGER before_crime_date_check BEFORE INSERT ON crimes FOR EACH ROW BEGIN IF NEW.crime_date > CURDATE() THEN SIGNAL SQLSTATE '45000' SET
MESSAGE_TEXT = 'Дата преступления не может быть в будущем'; END IF; END;
```

[Построение редактирование] [Изменить] [Создать PHP-код]

2. Триггер для автоматического обновления статуса преступника при аресте

```
DELIMITER //
```

```
CREATE TRIGGER after_imprisonment_insert
AFTER INSERT ON imprisonments
FOR EACH ROW
BEGIN
```

```
IF NEW.status = 'Отбывает' THEN

    UPDATE criminals

    SET status = 'Арестован'

    WHERE id = NEW.criminal_id;

END IF;

END//

DELIMITER ;
```

```
✓ MySQL вернула пустой результат (т.е. ноль строк). (Запрос занял 0,0083 сек.)

CREATE TRIGGER after_imprisonment_insert AFTER INSERT ON imprisonments FOR EACH ROW BEGIN IF NEW.status = 'Отбывает' THEN UPDATE criminals SET status = 'Арестован' WHERE id = NEW.criminal_id; END IF; END;

[ Построчное редактирование ] [ Изменить ] [ Создать PHP-код ]
```

3. Триггер для логирования изменений в таблице преступников

-- Сначала создадим таблицу для логов

```
CREATE TABLE criminal_audit_log (

    id INT PRIMARY KEY AUTO_INCREMENT,

    criminal_id INT NOT NULL,

    action VARCHAR(10) NOT NULL,

    changed_column VARCHAR(50),

    old_value TEXT,

    new_value TEXT,

    changed_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,

    changed_by VARCHAR(100)

);

DELIMITER //
```

```

CREATE TRIGGER after_criminal_update
AFTER UPDATE ON criminals
FOR EACH ROW
BEGIN
    -- Логируем изменение статуса
    IF OLD.status != NEW.status THEN
        INSERT INTO criminal_audit_log
        (criminal_id, action, changed_column, old_value, new_value, changed_by)
        VALUES
        (NEW.id, 'UPDATE', 'status', OLD.status, NEW.status, USER());
    END IF;

    -- Логируем изменение псевдонима
    IF OLD.alias != NEW.alias THEN
        INSERT INTO criminal_audit_log
        (criminal_id, action, changed_column, old_value, new_value, changed_by)
        VALUES
        (NEW.id, 'UPDATE', 'alias', OLD.alias, NEW.alias, USER());
    END IF;
END//

DELIMITER ;

```

✓ MySQL вернула пустой результат (т.е. ноль строк). (Запрос занял 0,0020 сек.)

```
-- Сначала создадим таблицу для логов CREATE TABLE criminal_audit_log ( id INT PRIMARY KEY AUTO_INCREMENT, criminal_id INT NOT NULL, action VARCHAR(10) NOT NULL, changed_column VARCHAR(50), old_value TEXT, new_value TEXT, changed_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP, changed_by VARCHAR(100) );
```

[Построчное редактирование] [Изменить] [Создать PHP-код]

✓ MySQL вернула пустой результат (т.е. ноль строк). (Запрос занял 0,0061 сек.)

```
CREATE TRIGGER after_criminal_update AFTER UPDATE ON criminals FOR EACH ROW BEGIN -- Логируем изменение статуса IF OLD.status != NEW.status THEN INSERT INTO criminal_audit_log (criminal_id, action, changed_column, old_value, new_value, changed_by) VALUES (NEW.id, 'UPDATE', 'status', OLD.status, NEW.status, USER()); END IF; -- Логируем изменение псевдонима IF OLD.alias != NEW.alias THEN INSERT INTO criminal_audit_log (criminal_id, action, changed_column, old_value, new_value, changed_by) VALUES (NEW.id, 'UPDATE', 'alias', OLD.alias, NEW.alias, USER()); END IF; END;
```

[Построчное редактирование] [Изменить] [Создать PHP-код]

4. Триггер для проверки дат заключения

DELIMITER //

CREATE TRIGGER before_imprisonment_date_check

BEFORE INSERT ON imprisonments

FOR EACH ROW

BEGIN

-- Проверка, что дата начала не в будущем

IF NEW.start_date > CURDATE() THEN

SIGNAL SQLSTATE '45000'

SET MESSAGE_TEXT = 'Дата начала заключения не может быть в будущем';

END IF;

-- Проверка, что дата окончания не раньше даты начала

IF NEW.end_date IS NOT NULL AND NEW.end_date < NEW.start_date THEN

SIGNAL SQLSTATE '45000'

SET MESSAGE_TEXT = 'Дата окончания не может быть раньше даты начала';

END IF;

END//

DELIMITER ;

✓ MySQL вернула пустой результат (т.е. ноль строк). (Запрос занял 0,0076 сек.)

```
CREATE TRIGGER before_imprisonment_date_check BEFORE INSERT ON imprisonments FOR EACH ROW BEGIN -- Проверка, что дата начала не в будущем IF
NEW.start_date > CURDATE() THEN SIGNAL SQLSTATE '45000' SET MESSAGE_TEXT = 'Дата начала заключения не может быть в будущем'; END IF; -- Проверка, что
дата окончания не раньше даты начала IF NEW.end_date IS NOT NULL AND NEW.end_date < NEW.start_date THEN SIGNAL SQLSTATE '45000' SET MESSAGE_TEXT = 'Дата
окончания не может быть раньше даты начала'; END IF; END;
```

[Построчное редактирование] [Изменить] [Создать PHP-код]

5. Триггер для автоматической установки даты создания

```
DELIMITER //
```

```
CREATE TRIGGER before_criminal_insert  
BEFORE INSERT ON criminals  
FOR EACH ROW  
BEGIN  
    -- Автоматическая установка даты создания, если не указана  
    IF NEW.created_at IS NULL THEN  
        SET NEW.created_at = CURRENT_TIMESTAMP;  
    END IF;  
END//
```

```
DELIMITER ;
```

✓ MySQL вернула пустой результат (т.е. ноль строк). (Запрос занял 0,0082 сек.)

```
CREATE TRIGGER before_criminal_insert BEFORE INSERT ON criminals FOR EACH ROW BEGIN -- Автоматическая установка даты создания, если не указана IF  
NEW.created_at IS NULL THEN SET NEW.created_at = CURRENT_TIMESTAMP; END IF; END;
```

[Построчное редактирование](#) [\[\[Изменить \]\]](#) [Создать PHP-код](#)

6. Триггер для проверки возраста жертвы

```
DELIMITER //
```

```
CREATE TRIGGER before_victim_insert  
BEFORE INSERT ON victims  
FOR EACH ROW  
BEGIN  
    DECLARE victim_age INT;
```

```

-- Вычисляем возраст жертвы
SET victim_age = TIMESTAMPDIFF(YEAR, NEW.birth_date, CURDATE());

-- Проверяем, что возраст реалистичный (между 0 и 120 лет)
IF victim_age < 0 OR victim_age > 120 THEN
    SIGNAL SQLSTATE '45000'
    SET MESSAGE_TEXT = 'Некорректная дата рождения жертвы';
END IF;
END//

DELIMITER ;

```

✓ MySQL вернула пустой результат (т.е. ноль строк). (Запрос занял 0,0076 сек.)

```

CREATE TRIGGER before_victim_insert BEFORE INSERT ON victims FOR EACH ROW BEGIN DECLARE victim_age INT; -- Вычисляем возраст жертвы SET victim_age =
TIMESTAMPDIFF(YEAR, NEW.birth_date, CURDATE()); -- Проверяем, что возраст реалистичный (между 0 и 120 лет) IF victim_age < 0 OR victim_age > 120 THEN
SIGNAL SQLSTATE '45000' SET MESSAGE_TEXT = 'Некорректная дата рождения жертвы'; END IF; END;

```

Построчное редактирование [\[\[Изменить \]\]](#) [\[\[Создать PHP-код \]\]](#)

Триггеры

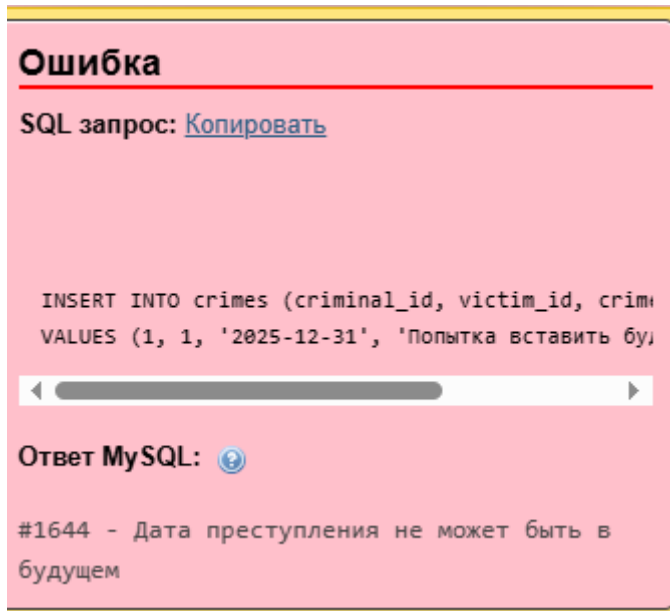
☐ Отметить все
 [Экспорт](#)
[Удалить](#)

	Имя	Таблица	Время	Событие	
☐	after_criminal_update	criminals	AFTER	UPDATE	Изменить Экспорт Удалить
☐	after_imprisonment_insert	imprisonments	AFTER	INSERT	Изменить Экспорт Удалить
☐	before_crime_date_check	crimes	BEFORE	INSERT	Изменить Экспорт Удалить
☐	before_criminal_insert	criminals	BEFORE	INSERT	Изменить Экспорт Удалить
☐	before_imprisonment_date_check	imprisonments	BEFORE	INSERT	Изменить Экспорт Удалить
☐	before_victim_insert	victims	BEFORE	INSERT	Изменить Экспорт Удалить

Проверка работы триггеров:

Тест триггера проверки даты преступления:

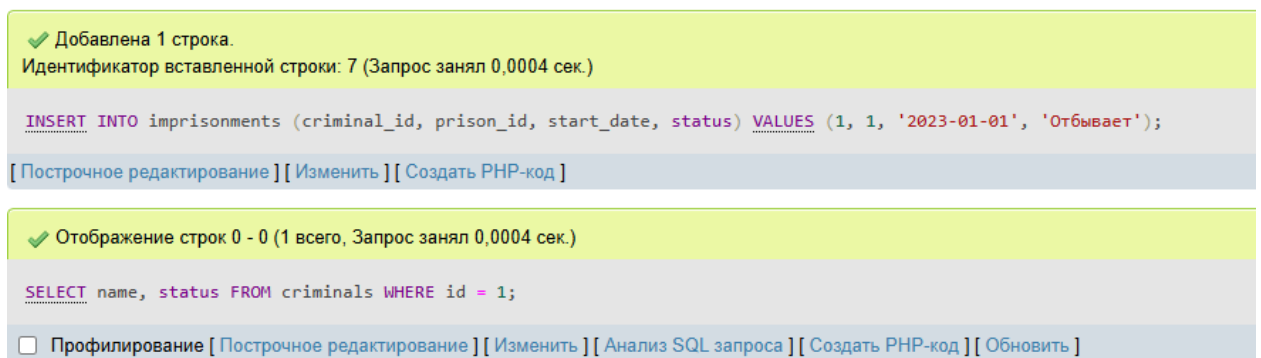
```
INSERT INTO crimes (criminal_id, victim_id, crime_date, description)
VALUES (1, 1, '2025-12-31', 'Попытка вставить будущую дату');
```



Тест триггера обновления статуса:

```
INSERT INTO imprisonments (criminal_id, prison_id, start_date, status)
VALUES (1, 1, '2023-01-01', 'Отбывает');
```

```
SELECT name, status FROM criminals WHERE id = 1;
```



Тест триггера логирования:

```
UPDATE criminals
SET status = 'Беглец'
WHERE id = 1;
```

```
SELECT * FROM criminal_audit_log;
```

✓ Затронута 1 строка. (Запрос занял 0,0006 сек.)

```
UPDATE criminals SET status = 'Беглец' WHERE id = 1;
```

[Построчное редактирование] [Изменить] [Создать PHP-код]

✓ Отображение строк 0 - 0 (1 всего, Запрос занял 0,0004 сек.)

```
SELECT * FROM criminal_audit_log;
```

☐ Профилирование [Построчное редактирование] [Изменить] [Анализ SQL запроса] [Создать PHP-код] [Обновить]

Хранимые процедуры

1. Хранимая процедура: Поиск преступлений по периоду

DELIMITER //

```
CREATE PROCEDURE GetCrimesByPeriod(  
    IN start_date DATE,  
    IN end_date DATE  
)  
BEGIN  
    SELECT  
        c.name AS criminal_name,  
        v.name AS victim_name,  
        cr.crime_date,  
        cr.description,  
        cs.city AS crime_city  
    FROM crimes cr  
    JOIN criminals c ON cr.criminal_id = c.id  
    JOIN victims v ON cr.victim_id = v.id  
    LEFT JOIN crime_scenes cs ON cr.id = cs.crime_id  
    WHERE cr.crime_date BETWEEN start_date AND end_date  
    ORDER BY cr.crime_date DESC;  
END//  
  
DELIMITER ;
```

✓ MySQL вернула пустой результат (т.е. ноль строк). (Запрос занял 0,0109 сек.)

```
CREATE PROCEDURE GetCrimesByPeriod( IN start_date DATE, IN end_date DATE ) BEGIN SELECT c.name AS criminal_name, v.name AS victim_name, cr.crime_date,  
cr.description, cs.city AS crime_city FROM crimes cr JOIN criminals c ON cr.criminal_id = c.id JOIN victims v ON cr.victim_id = v.id LEFT JOIN  
crime_scenes cs ON cr.id = cs.crime_id WHERE cr.crime_date BETWEEN start_date AND end_date ORDER BY cr.crime_date DESC; END;
```

[Построчное редактирование](#) | [Изменить](#) | [Создать PHP-код](#)

2. Хранимая процедура: Добавление нового преступления

DELIMITER //

```
CREATE PROCEDURE AddNewCrime(
```

```
    IN criminal_name VARCHAR(100),
```

```
    IN victim_name VARCHAR(100),
```

```
    IN crime_date DATE,
```

```
    IN crime_description TEXT,
```

```
    IN crime_city VARCHAR(50),
```

```
    IN crime_address TEXT
```

```
)
```

```
BEGIN
```

```
    DECLARE criminal_id INT;
```

```
    DECLARE victim_id INT;
```

```
    DECLARE new_crime_id INT;
```

```
    DECLARE EXIT HANDLER FOR SQLEXCEPTION
```

```
    BEGIN
```

```
        ROLLBACK;
```

```
        RESIGNAL;
```

```
    END;
```

```
    START TRANSACTION;
```

```
    -- Находим ID преступника
```

```
    SELECT id INTO criminal_id FROM criminals WHERE name =  
criminal_name;
```

```
    IF criminal_id IS NULL THEN
```

```
        SIGNAL SQLSTATE '45000' SET MESSAGE_TEXT = 'Преступник не  
найден';
```

```

END IF;

-- Находим или создаем жертву
SELECT id INTO victim_id FROM victims WHERE name = victim_name;
IF victim_id IS NULL THEN
    INSERT INTO victims (name, gender) VALUES (victim_name,
'Неизвестно');
    SET victim_id = LAST_INSERT_ID();
END IF;

-- Добавляем преступление
INSERT INTO crimes (criminal_id, victim_id, crime_date, description)
VALUES (criminal_id, victim_id, crime_date, crime_description);

SET new_crime_id = LAST_INSERT_ID();

-- Добавляем место преступления
INSERT INTO crime_scenes (crime_id, city, address)
VALUES (new_crime_id, crime_city, crime_address);

COMMIT;

SELECT CONCAT('Преступление успешно добавлено. ID: ', new_crime_id)
AS result;
END//

DELIMITER ;

```

✓ MySQL вернула пустой результат (т.е. ноль строк). (Запрос занял 0.0087 сек.)

```

CREATE PROCEDURE AddNewCrime( IN criminal_name VARCHAR(100), IN victim_name VARCHAR(100), IN crime_date DATE, IN crime_description TEXT, IN crime_city
VARCHAR(50), IN crime_address TEXT ) BEGIN DECLARE criminal_id INT; DECLARE victim_id INT; DECLARE new_crime_id INT; DECLARE EXIT HANDLER FOR
SQLEXCEPTION BEGIN ROLLBACK; RESIGNAL; END; START TRANSACTION; -- Находим ID преступника SELECT id INTO criminal_id FROM criminals WHERE name =
criminal_name; IF criminal_id IS NULL THEN SIGNAL SQLSTATE '45000' SET MESSAGE_TEXT = 'Преступник не найден'; END IF; -- Находим или создаем жертву
SELECT id INTO victim_id FROM victims WHERE name = victim_name; IF victim_id IS NULL THEN INSERT INTO victims (name, gender) VALUES (victim_name,
'Неизвестно'); SET victim_id = LAST_INSERT_ID(); END IF; -- Добавляем преступление [...]

```

[Изменить]

3. Функция: Расчет общего количества жертв преступника

DELIMITER //

```
CREATE FUNCTION GetVictimCount(criminal_id_param INT)
```

```
RETURNS INT
```

```
READS SQL DATA
```

```
DETERMINISTIC
```

```
BEGIN
```

```
    DECLARE victim_count INT;
```

```
    SELECT COUNT(*) INTO victim_count
```

```
    FROM crimes
```

```
    WHERE criminal_id = criminal_id_param;
```

```
    RETURN victim_count;
```

```
END//
```

DELIMITER ;

```
✓ MySQL вернула пустой результат (т.е. ноль строк). (Запрос занял 0,0045 сек.)

CREATE FUNCTION GetVictimCount(criminal_id_param INT) RETURNS INT READS SQL DATA DETERMINISTIC BEGIN DECLARE victim_count INT; SELECT COUNT(*) INTO victim_count FROM crimes WHERE criminal_id = criminal_id_param; RETURN victim_count; END;

[Построение графика] [Изменить] [Создать PHP-код]
```

4. Хранимая процедура: Статистика по преступникам

DELIMITER //

DELIMITER //

```
CREATE PROCEDURE GetCriminalStatistics()
```

```
BEGIN
```

```

SELECT
    c.name AS criminal_name,
    c.status,
    COUNT(cr.id) AS total_crimes,
    GetVictimCount(c.id) AS victim_count,
    pp.danger_level,
    p.name AS current_prison
FROM criminals c
LEFT JOIN crimes cr ON c.id = cr.criminal_id
LEFT JOIN psychological_profiles pp ON c.id = pp.criminal_id
LEFT JOIN imprisonments i ON c.id = i.criminal_id AND i.status = 'Отбывает'
LEFT JOIN prisons p ON i.prison_id = p.id
GROUP BY c.id, c.name, c.status, pp.danger_level, p.name
ORDER BY total_crimes DESC;
END//

```

DELIMITER ;

✓ MySQL вернула пустой результат (т.е. ноль строк). (Запрос занял 0,0081 сек.)

```

CREATE PROCEDURE GetCriminalStatistics() BEGIN SELECT c.name AS criminal_name, c.status, COUNT(cr.id) AS total_crimes, GetVictimCount(c.id) AS victim_count, pp.danger_level, p.name AS current_prison FROM criminals c LEFT JOIN crimes cr ON c.id = cr.criminal_id LEFT JOIN psychological_profiles pp ON c.id = pp.criminal_id LEFT JOIN imprisonments i ON c.id = i.criminal_id AND i.status = 'Отбывает' LEFT JOIN prisons p ON i.prison_id = p.id GROUP BY c.id, c.name, c.status, pp.danger_level, p.name ORDER BY total_crimes DESC; END;

```

Построчное редактирование [\[\[Изменить \]\]](#) [\[\[Создать PHP-код \]\]](#)

5. Функция: Определение уровня опасности преступника

DELIMITER //

```

CREATE FUNCTION GetCriminalDangerLevel(criminal_id_param INT)
RETURNS VARCHAR(20)
READS SQL DATA
DETERMINISTIC

```

```
BEGIN
```

```
DECLARE danger VARCHAR(20);
```

```
DECLARE crime_count INT;
```

```
-- Считаем количество преступлений
```

```
SELECT COUNT(*) INTO crime_count
```

```
FROM crimes
```

```
WHERE criminal_id = criminal_id_param;
```

```
-- Определяем уровень опасности на основе количества преступлений
```

```
IF crime_count >= 10 THEN
```

```
    SET danger = 'Критический';
```

```
ELSEIF crime_count >= 5 THEN
```

```
    SET danger = 'Высокий';
```

```
ELSEIF crime_count >= 2 THEN
```

```
    SET danger = 'Средний';
```

```
ELSE
```

```
    SET danger = 'Низкий';
```

```
END IF;
```

```
RETURN danger;
```

```
END//
```

```
DELIMITER ;
```

✓ MySQL вернула пустой результат (т.е. ноль строк). (Запрос занял 0,0055 сек.)

```
CREATE FUNCTION GetCriminalDangerLevel(criminal_id_param INT) RETURNS VARCHAR(20) READS SQL DATA DETERMINISTIC BEGIN DECLARE danger VARCHAR(20); DECLARE crime_count INT; -- Считаем количество преступлений SELECT COUNT(*) INTO crime_count FROM crimes WHERE criminal_id = criminal_id_param; -- Определяем уровень опасности на основе количества преступлений IF crime_count >= 10 THEN SET danger = 'Критический'; ELSEIF crime_count >= 5 THEN SET danger = 'Высокий'; ELSEIF crime_count >= 2 THEN SET danger = 'Средний'; ELSE SET danger = 'Низкий'; END IF; RETURN danger; END;
```

[\[Построение редактирование \]](#) [\[Изменить \]](#) [\[Создать PHP-код \]](#)

Примеры использования:

Использование процедуры поиска преступлений:

```
CALL GetCrimesByPeriod('2023-01-01', '2024-12-31');
```

✓ MySQL вернула пустой результат (т.е. ноль строк). (Запрос занял 0,0042 сек.)

```
CALL GetCrimesByPeriod('2023-01-01', '2024-12-31');
```

[Построчное редактирование](#) [\[Изменить \]](#) [\[Создать РНР-код \]](#)

Использование процедуры добавления преступления:

```
CALL AddNewCrime(  
    'Теодор Качинский',  
    'Новая Жертва',  
    '2024-01-15',  
    'Взрыв в университетском кампусе',  
    'Чикаго',  
    'Университетский проспект, 123'  
);
```

✓ Отображение строк 0 - 0 (1 всего, Запрос занял 0,0071 сек.)

```
CALL AddNewCrime( 'Теодор Качинский', 'Новая Жертва', '2024-01-15', 'Взрыв в университетском кампусе', 'Чикаго', 'Университетский проспект, 123' );
```

[\[Построчное редактирование \]](#) [\[Изменить \]](#) [\[Создать РНР-код \]](#)

☐ Показать все | Количество строк: 25 | Фильтровать строки:

Extra options

result

Преступление успешно добавлено. ID: 10

Использование функции подсчета жертв:

```
SELECT
```

```
    name,
```

```
GetVictimCount(id) AS victim_count
FROM criminals
WHERE id = 1;
```

✓ Отображение строк 0 - 0 (1 всего, Запрос занял 0,0049 сек.)

```
SELECT name, GetVictimCount(id) AS victim_count FROM criminals WHERE id = 1;
```

☐ Профилирование [\[Построчное редактирование \]](#) [\[Изменить \]](#) [\[Анализ SQL запроса \]](#) [\[Создать PHP-код \]](#) [\[Обновить \]](#)

☐ Показать все | Количество строк: 25 ▼ | Фильтровать строки:

Extra options

	name	victim_count
☐ Изменить Копировать Удалить	Теодор Качинский	2

☐ Отметить все | С отмеченными: Изменить Копировать Удалить Экспорт

Использование процедуры статистики:

```
CALL GetCriminalStatistics();
```

✓ Отображение строк 0 - 5 (6 всего, Запрос занял 0,0045 сек.)

```
CALL GetCriminalStatistics();
```

[\[Построчное редактирование \]](#) [\[Изменить \]](#) [\[Создать PHP-код \]](#)

☐ Показать все | Количество строк: 25 ▼ | Фильтровать строки:

Extra options

criminal_name	status	total_crimes	victim_count	danger_level	current_prison
Теодор Качинский	Беглец	4	2	Высокий	Тюрьма ADX Флоренс
Андрей Чикатило	Умер	2	2	Критический	NULL
Джеффри Дамер	Умер	2	2	Критический	NULL
Чарльз Мэнсон	Умер	1	1	Высокий	NULL
Джон Уэйн Гейси	Умер	1	1	Высокий	NULL
Ричард Рамирес	Умер	1	1	Критический	NULL

Использование функции уровня опасности:

```
SELECT
    name,
    GetCriminalDangerLevel(id) AS calculated_danger_level
```

FROM criminals;

SELECT name, GetCriminalDangerLevel(id) AS calculated_danger_level FROM criminals;

☐ Профилирование
 ☒ Построчное редактирование
 ☐ Изменить
 ☐ Анализ SQL запроса
 ☐ Создать PHP-код
 ☐ Обновить

☐ Показать все
 |
 Количество строк:
 Фильтровать строки:
 Сортировать по ключу:

Extra options

	name	calculated_danger_level
☐ Изменить Копировать Удалить	Теодор Качинский	Средний
☐ Изменить Копировать Удалить	Джеффри Дамер	Средний
☐ Изменить Копировать Удалить	Андрей Чикатило	Средний
☐ Изменить Копировать Удалить	Чарльз Мэнсон	Низкий
☐ Изменить Копировать Удалить	Джон Уэйн Гейси	Низкий
☐ Изменить Копировать Удалить	Ричард Рамирес	Низкий

Процедуры

☐ Отметить все	 Экспорт	 Удалить
---------------------------------------	---	---

	Имя	Тип	Возвращает	
☐	AddNewCrime	PROCEDURE		 Изменить  Выполнить  Экспорт  Удалить
☐	GetCrimesByPeriod	PROCEDURE		 Изменить  Выполнить  Экспорт  Удалить
☐	GetCriminalDangerLevel	FUNCTION	varchar(20)	 Изменить  Выполнить  Экспорт  Удалить
☐	GetCriminalStatistics	PROCEDURE		 Изменить  Выполнить  Экспорт  Удалить
☐	GetVictimCount	FUNCTION	int(11)	 Изменить  Выполнить  Экспорт  Удалить

Доступ к БД должны иметь не менее 3-х пользователей с разными правами (кроме root).

1. Создание пользователей и назначение прав

```
CREATE USER 'detective'@'localhost' IDENTIFIED BY '123';
```

```
CREATE USER 'analyst'@'localhost' IDENTIFIED BY '234';
```

```
CREATE USER 'operator'@'localhost' IDENTIFIED BY '345';
```

-- 1. Detective - ТОЛЬКО ЧТЕНИЕ определенных таблиц

```
GRANT SELECT ON criminal_dossier.criminals TO 'detective'@'localhost';
```

```
GRANT SELECT ON criminal_dossier.crimes TO 'detective'@'localhost';
```

```
GRANT SELECT ON criminal_dossier.victims TO 'detective'@'localhost';
```

```
GRANT SELECT ON criminal_dossier.crime_scenes TO 'detective'@'localhost';
```

```
GRANT EXECUTE ON PROCEDURE criminal_dossier.GetCrimesByPeriod TO 'detective'@'localhost';
```

-- 2. Analyst - ЧТЕНИЕ всех таблиц + процедуры

```
GRANT SELECT ON criminal_dossier.* TO 'analyst'@'localhost';
```

```
GRANT EXECUTE ON PROCEDURE criminal_dossier.GetCrimesByPeriod TO 'analyst'@'localhost';
```

```
GRANT EXECUTE ON PROCEDURE criminal_dossier.GetCriminalStatistics TO 'analyst'@'localhost';
```

```
GRANT EXECUTE ON FUNCTION criminal_dossier.GetVictimCount TO 'analyst'@'localhost';
```

```
GRANT EXECUTE ON FUNCTION criminal_dossier.GetCriminalDangerLevel TO 'analyst'@'localhost';
```

-- 3. Operator - ЧТЕНИЕ + ЗАПИСЬ определенных таблиц

```
GRANT SELECT, INSERT, UPDATE ON criminal_dossier.criminals TO 'operator'@'localhost';
```

GRANT SELECT, INSERT, UPDATE ON criminal_dossier.victims TO
'operator'@'localhost';

GRANT SELECT, INSERT, UPDATE ON criminal_dossier.crimes TO
'operator'@'localhost';

GRANT SELECT, INSERT, UPDATE ON criminal_dossier.crime_scenes TO
'operator'@'localhost';

GRANT SELECT, INSERT, UPDATE ON criminal_dossier.imprisonments TO
'operator'@'localhost';

GRANT EXECUTE ON PROCEDURE criminal_dossier.AddNewCrime TO
'operator'@'localhost';

REVOKE ALL PRIVILEGES ON *.* FROM 'detective'@'localhost';

REVOKE ALL PRIVILEGES ON *.* FROM 'analyst'@'localhost';

REVOKE ALL PRIVILEGES ON *.* FROM 'operator'@'localhost';

FLUSH PRIVILEGES;

✓ MySQL вернула пустой результат (т.е. ноль строк). (Запрос занял 0,0117 сек.)

```
-- Пользователь 1: Детектив (только чтение основных таблиц) CREATE USER 'detective'@'localhost' IDENTIFIED BY 'detective_pass123';
```

[Построчное редактирование](#) [\[\[Изменить \]\]](#) [\[\[Создать PHP-код \]\]](#)

✓ MySQL вернула пустой результат (т.е. ноль строк). (Запрос занял 0,0696 сек.)

```
GRANT SELECT ON criminal_dossier.criminals TO 'detective'@'localhost';
```

[Построчное редактирование](#) [\[\[Изменить \]\]](#) [\[\[Создать PHP-код \]\]](#)

✓ MySQL вернула пустой результат (т.е. ноль строк). (Запрос занял 0,0150 сек.)

```
GRANT SELECT ON criminal_dossier.crimes TO 'detective'@'localhost';
```

[Построчное редактирование](#) [\[\[Изменить \]\]](#) [\[\[Создать PHP-код \]\]](#)

✓ MySQL вернула пустой результат (т.е. ноль строк). (Запрос занял 0,0129 сек.)

```
GRANT SELECT ON criminal_dossier.victims TO 'detective'@'localhost';
```

[Построчное редактирование](#) [\[\[Изменить \]\]](#) [\[\[Создать PHP-код \]\]](#)

✓ MySQL вернула пустой результат (т.е. ноль строк). (Запрос занял 0,0251 сек.)

```
GRANT SELECT ON criminal_dossier.crime_scenes TO 'detective'@'localhost';
```

[Построчное редактирование] [Изменить] [Создать PHP-код]

✓ MySQL вернула пустой результат (т.е. ноль строк). (Запрос занял 0,0143 сек.)

```
GRANT EXECUTE ON PROCEDURE criminal_dossier.GetCrimesByPeriod TO 'detective'@'localhost';
```

[Построчное редактирование] [Изменить] [Создать PHP-код]

✓ MySQL вернула пустой результат (т.е. ноль строк). (Запрос занял 0,0136 сек.)

```
GRANT EXECUTE ON FUNCTION criminal_dossier.GetVictimCount TO 'detective'@'localhost';
```

[Построчное редактирование] [Изменить] [Создать PHP-код]

✓ MySQL вернула пустой результат (т.е. ноль строк). (Запрос занял 0,0081 сек.)

```
-- Пользователь 2: Аналитик (чтение + выполнение процедур) CREATE USER 'analyst'@'localhost' IDENTIFIED BY 'analyst_pass123';
```

[Построчное редактирование] [Изменить] [Создать PHP-код]

✓ MySQL вернула пустой результат (т.е. ноль строк). (Запрос занял 0,0022 сек.)

```
GRANT SELECT ON criminal_dossier.* TO 'analyst'@'localhost';
```

[Построчное редактирование] [Изменить] [Создать PHP-код]

✓ MySQL вернула пустой результат (т.е. ноль строк). (Запрос занял 0,0016 сек.)

```
GRANT EXECUTE ON criminal_dossier.* TO 'analyst'@'localhost';
```

[Построчное редактирование] [Изменить] [Создать PHP-код]

✓ MySQL вернула пустой результат (т.е. ноль строк). (Запрос занял 0,0010 сек.)

```
GRANT SHOW VIEW ON criminal_dossier.* TO 'analyst'@'localhost';
```

[Построчное редактирование] [Изменить] [Создать PHP-код]

✓ MySQL вернула пустой результат (т.е. ноль строк). (Запрос занял 0,0011 сек.)

```
-- Пользователь 3: Оператор (чтение + вставка + обновление) CREATE USER 'operator'@'localhost' IDENTIFIED BY 'operator_pass123';
```

[Построчное редактирование] [Изменить] [Создать PHP-код]

✓ MySQL вернула пустой результат (т.е. ноль строк). (Запрос занял 0,0012 сек.)

```
GRANT SELECT, INSERT, UPDATE ON criminal_dossier.criminals TO 'operator'@'localhost';
```

[Построчное редактирование] [Изменить] [Создать PHP-код]

✓ MySQL вернула пустой результат (т.е. ноль строк). (Запрос занял 0,0014 сек.)

```
GRANT SELECT, INSERT, UPDATE ON criminal_dossier.victims TO 'operator'@'localhost';
```

[Построчное редактирование] [Изменить] [Создать PHP-код]

✓ MySQL вернула пустой результат (т.е. ноль строк). (Запрос занял 0,0013 сек.)

```
GRANT SELECT, INSERT, UPDATE ON criminal_dossier.crimes TO 'operator'@'localhost';
```

[Построчное редактирование] [Изменить] [Создать PHP-код]

✓ MySQL вернула пустой результат (т.е. ноль строк). (Запрос занял 0,0009 сек.)

```
GRANT SELECT, INSERT, UPDATE ON criminal_dossier.crime_scenes TO 'operator'@'localhost';
```

[Построчное редактирование] [Изменить] [Создать PHP-код]

```
✓ MySQL вернула пустой результат (т.е. ноль строк). (Запрос занял 0,0012 сек.)

GRANT SELECT, INSERT, UPDATE ON criminal_dossier.imprisonments TO 'operator'@'localhost';

[ Построчное редактирование ] [ Изменить ] [ Создать PHP-код ]

✓ MySQL вернула пустой результат (т.е. ноль строк). (Запрос занял 0,0019 сек.)

GRANT EXECUTE ON PROCEDURE criminal_dossier.AddNewCrime TO 'operator'@'localhost';

[ Построчное редактирование ] [ Изменить ] [ Создать PHP-код ]

✓ MySQL вернула пустой результат (т.е. ноль строк). (Запрос занял 0,0022 сек.)

GRANT EXECUTE ON PROCEDURE criminal_dossier.GetCriminalStatistics TO 'operator'@'localhost';

[ Построчное редактирование ] [ Изменить ] [ Создать PHP-код ]

✓ MySQL вернула пустой результат (т.е. ноль строк). (Запрос занял 0,0022 сек.)

-- Применяем изменения прав FLUSH PRIVILEGES;

[ Построчное редактирование ] [ Изменить ] [ Создать PHP-код ]
```

Обзор учетных записей пользователей

Имя пользователя	Имя хоста	Пароль	Глобальные привилегии	Группа пользователей	Grant	Действие
☐ Любой	%	Нет	USAGE		Нет	Редактировать привилегии Экспорт Заблокировать
☐ analyst	localhost	Да	USAGE		Нет	Редактировать привилегии Экспорт Заблокировать
☐ detective	localhost	Да	USAGE		Нет	Редактировать привилегии Экспорт Заблокировать
☐ operator	localhost	Да	USAGE		Нет	Редактировать привилегии Экспорт Заблокировать
☐ pma	localhost	Нет	USAGE		Нет	Редактировать привилегии Экспорт Заблокировать
☐ root	127.0.0.1	Нет	ALL PRIVILEGES		Да	Редактировать привилегии Экспорт Заблокировать
☐ root	:::1	Нет	ALL PRIVILEGES		Да	Редактировать привилегии Экспорт Заблокировать
☐ root	localhost	Нет	ALL PRIVILEGES		Да	Редактировать привилегии Экспорт Заблокировать

☐ Отметить все С отмеченными: Экспорт

Пример запросов.

-- Сколько преступлений совершил каждый преступник?

SELECT

c.name AS criminal_name,

COUNT(cr.id) AS crimes_count,

c.status,

pp.danger_level

FROM criminals c

LEFT JOIN crimes cr ON c.id = cr.criminal_id

LEFT JOIN psychological_profiles pp ON c.id = pp.criminal_id

GROUP BY c.id

ORDER BY crimes_count DESC;

✔ Отображение строк 0 - 7 (8 всего, Запрос занял 0,0007 сек.) [crimes_count: 2... - 0...]

```
-- Сколько преступлений совершил каждый преступник? SELECT c.name AS criminal_name, COUNT(cr.id) AS crimes_count, c.status, pp.danger_level FROM criminals c LEFT JOIN crimes cr ON c.id = cr.criminal_id LEFT JOIN psychological_profiles pp ON c.id = pp.criminal_id GROUP BY c.id ORDER BY crimes_count DESC;
```

[Построенное редактирование] [Изменить] [Создать PHP-код]

☐ Показать все | Количество строк: 25 | Фильтровать строки: Поиск в таблице

Extra options

criminal_name	crimes_count	status	danger_level
Андрей Чикатило	2	Умер	Критический
Джеффри Дамер	2	Умер	Критический
Ричард Рамирес	1	Умер	Критический
Тед Банди	1	Беглец	Критический
Джон Уэйн Гейси	1	Умер	Высокий
Гэри Риджуэй	1	Активен	Низкий
Чарльз Мэнсон	1	Умер	Высокий
Теодор Качинский	0	Арестован	NULL

-- Покажем преступления с информацией о преступнике и жертве

SELECT

c.name AS criminal,

v.name AS victim,

cr.crime_date,

cr.description

FROM crimes cr

JOIN criminals c ON cr.criminal_id = c.id

JOIN victims v ON cr.victim_id = v.id

LIMIT 5;

✔ Отображение строк 0 - 4 (5 всего, Запрос занял 0,0006 сек.)

```
-- Покажем преступления с информацией о преступнике и жертве SELECT c.name AS criminal, v.name AS victim, cr.crime_date, cr.description FROM crimes cr JOIN criminals c ON cr.criminal_id = c.id JOIN victims v ON cr.victim_id = v.id LIMIT 5;
```

[Построенное редактирование] [Изменить] [Создать PHP-код]

Extra options

criminal	victim	crime_date	description
Джеффри Дамер	Лена Загортowa	1978-06-18	Первое убийство - автостопщик Стивен Хикс
Джеффри Дамер	Шэрон Тейт	1991-03-25	Убийство и расчленение в квартире
Андрей Чикатило	Роберт Пирст	1978-12-22	Первая доказанная жертва - Лариса Савенко
Андрей Чикатило	Мария Эрнандес	1982-09-15	Убийство в лесополосе
Чарльз Мэнсон	Джеймс Хьюстон	1969-08-09	Убийство Шэрон Тейт и гостей

Использование результатов запроса

Печать | В буфер обмена | Экспорт | Отобразить график | Создать представление

-- Распределение жертв по возрастным группам на момент преступления

SELECT

CASE

WHEN TIMESTAMPDIFF(YEAR, v.birth_date, cr.crime_date) < 18 THEN
'До 18'

WHEN TIMESTAMPDIFF(YEAR, v.birth_date, cr.crime_date) BETWEEN
18 AND 30 THEN '18-30'

WHEN TIMESTAMPDIFF(YEAR, v.birth_date, cr.crime_date) BETWEEN
31 AND 50 THEN '31-50'

ELSE 'Старше 50'

END AS age_group,

COUNT(*) AS victim_count,

v.gender

FROM crimes cr

JOIN victims v ON cr.victim_id = v.id

GROUP BY age_group, v.gender

ORDER BY victim_count DESC;

✔ Отображение строк 0 - 3 (4 всего, Запрос занял 0,0005 сек.)

-- Распределение жертв по возрастным группам на момент преступления SELECT CASE WHEN TIMESTAMPDIFF(YEAR, v.birth_date, cr.crime_date) < 18 THEN 'До 18' WHEN TIMESTAMPDIFF(YEAR, v.birth_date, cr.crime_date) BETWEEN 18 AND 30 THEN '18-30' WHEN TIMESTAMPDIFF(YEAR, v.birth_date, cr.crime_date) BETWEEN 31 AND 50 THEN '31-50' ELSE 'Старше 50' END AS age_group, COUNT(*) AS victim_count, v.gender FROM crimes cr JOIN victims v ON cr.victim_id = v.id GROUP BY age_group, v.gender ORDER BY victim_count DESC;

[Построчное редактирование] [Изменить] [Создать PHP-код]

☐ Показать все | Количество строк: 25 | Фильтровать строки: Поиск в таблице

Extra options

age_group	victim_count	1	gender
18-30	3	Мужской	
До 18	3	Женский	
18-30	2	Женский	
31-50	1	Женский	

Вывод

В ходе выполнения лабораторной работы была спроектирована и реализована база данных «Криминальная картотека: серийные преступники» на всех этапах проектирования